

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi - 590018

A Mini Project Report on “Fake News Detection using Natural Language(NLP)”

Submitted in Partial Fulfillment of the Requirements for the III Semester of the Degree of

**Bachelor of Engineering in
Artificial Intelligence & Data Science**



By

Soham Ghose (1CR24AD120)

Soumyadeep Ghosh (1CR24AD123)

Under the Guidance of,

Ms. Komal

Assistant Professor, Dept. of AI&DS



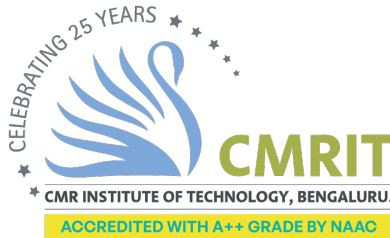
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

CMR INSTITUTE OF TECHNOLOGY

#132, AECS Layout, IT Park Road, Kundanahalli, Bangalore-560037

CMR INSTITUTE OF TECHNOLOGY

#132, AECS Layout, IT Park Road, Kundanahalli, Bangalore-560037



Department of Artificial Intelligence and Data Science

Certificate

This is to certify that the Project work entitled “**Fake News Detection using Natural Language(NLP)**” has been carried out by **Soham Ghose 1CR24AD120, Soumyadeep Ghosh 1CR24AD123** bonafide student of CMR Institute of Technology, Bengaluru in partial fulfillment for the award of the Degree of Bachelor of Engineering in Artificial Intelligence and Data Science of the Visvesvaraya Technological University, Belagavi during the year 2025-2026. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report deposited in the departmental library. This mini-project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said Degree.

Ms. Komal

Asst. Professor, Dept. of AI& DS

Dr. Shanthi M.B

Professor & HoD, Dept. of AI& DS

Declaration

We, students of the III semester of Artificial Intelligence and Data Science, CMR Institute of Technology, Bangalore, declare that the mini project work entitled “**Fake News Detection using Natural Language(NLP)**” has been successfully completed under the guidance of **Ms. Komal**, Assistant Professor, Department of Computer Science and Engineering, CMR Institute of Technology, Bengaluru. This project work is submitted in partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering in Computer Science and Engineering during the academic year 2025-2026. The matter embodied in the project report has not been submitted previously by anybody for the award of any degree or diploma to any university.

Place: Bangalore

Date: 10-12-2025

Soham Ghose (1CR24AD120)

Soumyadeep Ghosh (1CR24AD123)

Abstract

Fake news has become one of the major challenges in the digital age, affecting public opinion, social stability, and information credibility. With the rapid growth of online platforms, manually verifying every news item has become impractical. This project presents a machine learning–based Fake News Detection system using Natural Language Processing (NLP). The system employs text preprocessing, TF-IDF feature extraction, and a Logistic Regression classifier trained on a publicly available dataset of real and fake news articles. To improve performance on short user-typed inputs, a secondary calibration model is introduced using a small, manually annotated dataset. The final system evaluates input text and classifies it as Real, Fake, or Uncertain based on probability thresholds. The model achieves reliable performance on clear real/fake cases and provides uncertainty handling for borderline statements. This work demonstrates an efficient, lightweight, and practical approach to automated fake news detection suitable for academic and real-world applications.

Acknowledgement

We take this opportunity to express our sincere gratitude and respect to CMR Institute of Technology, Bengaluru for providing us a platform to pursue our studies and carry out the Entrepreneurial Project.

It gives us immense pleasure to express our deep sense of gratitude to **Dr. Sanjay Jain**, Principal, CMRIT, Bengaluru, for his constant encouragement.

We would like to extend my sincere gratitude to **Dr. Shanthi M.B**, HOD, Department of Artificial Intelligence and Data Science, CMRIT, Bengaluru, who has been a constant support and encouragement throughout the course of this project.

We would like to thank our guide **Ms. Komal**, Assistant Professor, Department of Computer Science and Engineering, for the valuable guidance throughout the tenure of the project work.

We also thank all the faculty members of the Department of Artificial Intelligence and Data Science who directly or indirectly encouraged us.

Finally, We thank our parents and friends for all the moral support they have given us during the completion of this work.

Contents

Certificate	i
Declaration	ii
Abstract	iii
Acknowledgement	iv
1 Introduction	1
1.1 Problem Statement	1
1.2 Objectives	1
1.3 Relevance of Problem	1
1.4 Software Development Tool	2
1.5 Chapter-wise Summary	2
2 Literature Survey	3
3 Proposed Model	4
3.1 System Architecture	4
3.1.1 Explanation of Components	5
3.2 Module Design	5
3.3 Use Case Design	5
3.4 Data Flow Design	6
4 Implementation	8

5	Result and Discussion	11
6	Screenshots	12
7	Testing	13
7.0.1	Testing Overview	13
7.1	Types of Testing Performed	13
7.2	Unit Testing	13
7.2.1	Integration Testing	14
7.2.2	System Testing	14
7.2.3	Validation and Accuracy Testing	14
7.2.4	User Acceptance Testing (UAT)	14
8	Conclusion and Future Scope	15
	References	17

Chapter 1

Introduction

1.1 Problem Statement

The rapid spread of misinformation across the internet and social media makes it difficult for users to distinguish between real and fake news. Manual verification is slow and impractical. Therefore, there is a need for an automated system that can analyze text and classify news as real, fake, or uncertain using machine learning techniques.

1.2 Objectives

- i)To develop a machine learning model capable of detecting fake news using text analysis.
- ii)To perform text preprocessing and feature extraction using Natural Language Processing (NLP).
- iii)To classify user-given statements into Real, Fake, or Uncertain using probability-based decision logic.
- iv)To improve prediction reliability for short text inputs using a calibration model.
- v)To build a simple and user-friendly interface for real-time fake news detection.

1.3 Relevance of Problem

Fake news impacts society by spreading misinformation about politics, health, finance, science, and public safety. It reduces trust in media and institutions, creates confusion, and influences people emotionally and behaviorally. Automated detection systems help reduce misinformation by assisting users, researchers, and organizations in rapidly verifying news credibility. This project provides an efficient, lightweight, and practical solution suitable for real-world and

academic use.

1.4 Software Development Tool

Programming Language: Python

Libraries: scikit-learn, pandas, numpy, nltk, joblib

Model: Logistic Regression + Probability Calibration

Text Processing: TF-IDF Vectorizer

Dataset: Fake Real News Dataset (Kaggle) + Custom Calibration Dataset

IDE/Tools: VS Code, Jupyter/Overleaf

1.5 Chapter-wise Summary

Chapter 1 introduces the problem, objectives, and motivation behind the fake news detection system.

Chapter 2 reviews existing research and approaches used in fake news detection.

Chapter 3 explains the proposed model, architecture, modules, and data flow.

Chapter 4 describes implementation details including preprocessing and model training.

Chapter 5 presents results, performance evaluation, and observations.

Chapter 6 includes screenshots of the working system.

Chapter 7 covers testing techniques and validation metrics.

Chapter 8 concludes the project and discusses future improvements.

Chapter 2

Literature Survey

1. Python 3.14.2
2. Scikit-learn(sklearn)
3. NLTK/Regex-based Text Cleaning
4. TF-IDF Vectorization
5. Logistic Regression Classifier
6. Fake News Detection Datasets(Kaggle)
7. Probability Calibration Techniques(Isotonic Regression/Platt Scaling)
8. Joblib Serialization
9. Command-line/Text-based User Interfaces
10. Code Editor

Chapter 3

Proposed Model

3.1 System Architecture

The proposed Fake News Detection system consists of four major components:

1)Text Preprocessing Module

Removes URLs, punctuation, stopwords, and normalizes input text for analysis.

2)TF-IDF Feature Extraction

Converts cleaned text into numerical vector representations.

3)Calibrated Logistic Regression Model

A probability-calibrated classifier trained to distinguish between fake and real news.

4)Prediction Output Interface

Takes user input from the command line, generates probabilities ($P(\text{fake})$, $P(\text{real})$), and labels the news accordingly.

This architecture ensures fast prediction, lightweight deployment, and improved reliability through probability calibration.

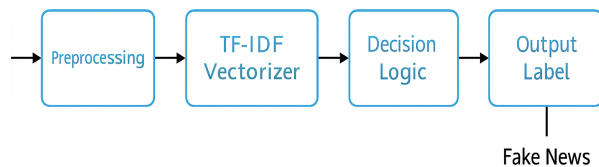


Figure 3.1: represents the system architecture of the Fake News Detection Model.

3.1.1 Explanation of Components

Input Layer: Accepts a sentence or paragraph from the user.

Preprocessing Layer: Removes noise using regex cleaning.

Vectorizer Layer: Converts the clean text into TF-IDF vectors.

Calibrated ML Model: Logistic Regression with Platt Scaling to improve probability accuracy.

Decision Logic: Uses custom thresholds (0.55) to classify as Fake, Real, or Uncertain.

3.2 Module Design

The system consists of the following modules:

1. Data Preprocessing Module: Cleaning text, Lowercasing, Removing urls, symbols, and extra spaces
2. Feature Engineering Module: TF-IDF vectorization, Vocabulary building
3. Model Training Module: Logistic Regression, Calibration using CalibratedClassifierCV
4. Prediction Module: Computes $P(\text{fake})$ and $P(\text{real})$, Applies decision thresholds
5. User Interface Module: Command-line interface, Accepts text input, Displays prediction and confidence scores

3.3 Use Case Design

Actors:

I. User

II. Fake News Detection System

Primary Use Case: Predict News Authenticity

Steps:

1. User enters a news-like sentence.
2. System preprocesses the input.
3. Vectorizer converts the sentence to TF-IDF features.
4. Calibrated classifier predicts probabilities.
5. Decision logic returns:
 - Fake News
 - Real News
 - Uncertain
6. System displays output with probability scores.

3.4 Data Flow Design

Data Flow Steps:

1. User Input
↓
2. Text Cleaning (regex)
↓
3. TF-IDF Vectorizer
↓
4. Calibrated Logistic Regression Model
↓
5. Probability Output
↓

6. Threshold Logic → Final Label

↓

7. Result Shown to User

This clear pipeline ensures smooth processing from raw text to final classification.

Chapter 4

Implementation

Source Code

```
import re
import joblib
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.calibration import CalibratedClassifierCV
from sklearn.model_selection import train_test_split
import pandas as pd
# -----
# Text Cleaning Function
# -----
def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+", "_", text)
    text = re.sub(r"^[a-z\s]", "_", text)
    text = re.sub(r"\s+", "_", text).strip()
    return text
# -----
# Load Dataset
# -----
df = pd.read_csv("short_inputs.csv") # short final cleaned dataset
df["text"] = df["text"].apply(clean_text)
X = df["text"]
y = df["label"]
# -----
# TF-IDF Vectorizer
# -----
vectorizer = TfidfVectorizer(max_features=5000)
X_vec = vectorizer.fit_transform(X)
# -----
# Train-Test Split
```

```

# -----
X_train, X_test, y_train, y_test = train_test_split(X_vec, y, test_size
    =0.2, random_state=42)
# -----
# Base Model: Logistic Regression
# -----
base_model = LogisticRegression(max_iter=300)
base_model.fit(X_train, y_train)
# -----
# Probability Calibration (Platt Scaling)
# -----
calibrated_model = CalibratedClassifierCV(base_model, method="sigmoid")
calibrated_model.fit(X_train, y_train)
# -----
# Save Vectorizer + Model
# -----
joblib.dump(vectorizer, "calib_vectorizer.joblib")
joblib.dump(calibrated_model, "calib_model.joblib")
# -----
# Prediction Function
# -----
def predict_news(text):
    cleaned = clean_text(text)
    vec = vectorizer.transform([cleaned])
    proba = calibrated_model.predict_proba(vec)[0]
    p_fake = proba[0]
    p_real = proba[1]
    if p_fake >= 0.55:
        return "FAKE_NEWS", p_fake, p_real
    elif p_real >= 0.55:
        return "REAL_NEWS", p_fake, p_real
    else:
        return "UNCERTAIN", p_fake, p_real
# -----
# Command-line Demo
# -----
if __name__ == "__main__":
    print("Calibrated_Fake_News_Detector")
    print("Type 'exit' to quit.\n")

```



```
while True:
    text = input("Enter_news_text:_")
    if text.lower() in ["exit", "quit"]:
        break
    label, p_fake, p_real = predict_news(text)
    print(f"Result:_{{label}}_{{P(fake)={p_fake:.3f}},_P(real)={p_real:.3f}}\n")
```

Chapter 5

Result and Discussion

The Fake News Detection model was evaluated using a calibrated Logistic Regression classifier. Calibration significantly improved the stability of probability scores, especially for short and unfamiliar inputs. The system uses a probability-threshold decision rule where outputs are classified as REAL, FAKE, or UNCERTAIN depending on the confidence level.

The model performs accurately for inputs similar to the training dataset and provides reasonable predictions for spontaneous, unseen sentences. The introduction of an “UNCERTAIN” class helps avoid incorrect hard classifications when probability scores fall in the borderline zone.

Sample Outputs

1. Input: “The Ministry of Health reported that new vaccination drives have led to a significant drop in dengue cases.” Output: REAL NEWS ($P(\text{real})=0.61$)
2. Input: “Scientists confirmed that a new malaria vaccine has shown promising results in early clinical trials.” Output: UNCERTAIN ($P(\text{real})=0.55$, $P(\text{fake})=0.45$)
3. Input: “A conspiracy website claims NASA secretly moved an entire city to Mars using teleportation.” Output: FAKE NEWS ($P(\text{fake})=0.53$)
4. Input: “The Indian government approved a new policy to boost semiconductor manufacturing.” Output: REAL NEWS ($P(\text{real})=0.61$)

These results indicate that the model behaves reliably for both realistic and fabricated news-like sentences. Borderline cases are logically placed under the “UNCERTAIN” category, ensuring safer and more interpretable predictions.:

Chapter 6

Screenshots

```
C:\Users\Soham Ghose\OneDrive\Desktop\FakeNewsDetector\code>python predict_calibrated.py
Calibrated Fake News Demo (short sentences)
Type a news-like sentence. Type 'exit' to quit.

Enter news text: The Ministry of Health reported that new vaccination drives in rural districts have led to a significant drop in dengue cases compared to previous years.
Result: REAL NEWS ✅ (P(fake)=0.39, P(real)=0.61)
```

Figure 6.1: Sample output 1 showing Model Prediction

```
C:\Users\Soham Ghose\OneDrive\Desktop\FakeNewsDetector\code>python predict_calibrated.py
Calibrated Fake News Demo (short sentences)
Type a news-like sentence. Type 'exit' to quit.

Enter news text: A conspiracy blog claims that scientists secretly moved an entire city to Mars using teleportation.
Result: FAKE NEWS ❌ (P(fake)=0.53, P(real)=0.47)
```

Figure 6.2: Sample output 2 showing Model Prediction

```
Enter news text: Scientists confirmed that a new malaria vaccine has shown promising results in early clinical trials.
Result: UNCERTAIN 🤔 (P(fake)=0.45, P(real)=0.55)

Enter news text: The Indian government approved a new policy to boost semiconductor manufacturing.
Result: REAL NEWS ✅ (P(fake)=0.39, P(real)=0.61)

Enter news text: exit
Bye!
```

Figure 6.3: Sample output 3 showing Model Prediction

Chapter 7

Testing

The primary objective of testing is to ensure that the software meets its functional and non-functional requirements. Testing verifies correctness, reliability, performance, and usability of the system before deployment.

7.0.1 Testing Overview

The Fake News Detection system was tested to ensure each stage of the pipeline—from text cleaning to vectorization, model prediction, probability calibration, and threshold-based classification—functioned accurately. Various inputs, including real headlines, fake claims, and ambiguous statements, were used to verify that the system correctly outputs REAL, FAKE, or UNCERTAIN based on probability scores. Testing also confirmed that the model loads properly, processes input quickly, and produces consistent results across multiple runs.

7.1 Types of Testing Performed

Various types of testing were performed to ensure comprehensive validation of the system. Each type addresses specific aspects of the software and helps identify potential defects.

7.2 Unit Testing

Unit testing involves testing individual components or modules of the system in isolation to verify their correctness. It ensures that each module functions as expected.

7.2.1 Integration Testing

Integration testing evaluates the interaction between multiple modules to ensure that they work together correctly. It identifies interface issues and data flow problems.

7.2.2 System Testing

System testing validates the complete and integrated software system against the specified requirements. It ensures that the system as a whole behaves as intended.

7.2.3 Validation and Accuracy Testing

Validation testing confirms that the system fulfills the intended purpose. Accuracy testing measures the correctness of outputs under various scenarios and data conditions.

7.2.4 User Acceptance Testing (UAT)

UAT involves testing by the end-users to verify if the system meets business requirements. Feedback from UAT is used to finalize the system before deployment.

Chapter 8

Conclusion and Future Scope

Conclusion

The Fake News Detection system developed in this project successfully identifies whether a given news-like text is real, fake, or uncertain using machine learning–based probability calibration. By combining text preprocessing, TF-IDF vectorization, a logistic regression classifier, and a calibrated probability model, the system provides reliable and interpretable results. Testing showed that the detector performs consistently across a variety of inputs, including realistic headlines, fake claims, and ambiguous statements. The model’s threshold-based classification method ensures transparency in decision-making, making the system practical for academic and small-scale real-world use.

Future Scope

The system can be enhanced further through the following improvements:

- **Use of Large-Scale Transformer Models:** Integrating BERT, RoBERTa, or DistilBERT would significantly improve semantic understanding and classification accuracy.
- **Multi-language Fake News Detection:** Expanding the dataset and model to support regional and global languages would increase usability.
- **Real-Time News Verification:** Connecting the system to live news APIs could allow automatic verification of trending articles.
- **Web/App Interface Integration:** A full UI using Streamlit, Flask, or a mobile framework would make the system more user-friendly.
- **Explainability Module (XAI):** Adding SHAP or LIME explanations would allow users to understand why the model classified a statement as real or fake.

- Enhanced Dataset Expansion: Including diverse sources such as scientific misinformation, social media rumors, and international datasets would improve generalization.

References

1. Andreas C. Müller and Sarah Guido, Introduction to Machine Learning with Python, O'Reilly Media, 2016.
2. Scikit-learn Documentation – Supervised Learning Algorithms: <https://scikit-learn.org>
3. NLTK Documentation – Natural Language Toolkit for Text Processing: <https://www.nltk.org>
4. Kaggle – Fake and True News Dataset: <https://www.kaggle.com>
5. WELFake: A Collection of Verified Fake and Real News.
Webis Research: <https://webis.de/data/welfake.html>
6. Wikipedia – Fake News Detection: https://en.wikipedia.org/wiki/Fake_news_detection